ITensor.jl & Matrix Product States (MPS)

Julia Command Reference & Syntax Guide

Author: Joaquín Peñuela Parra
Univ. of Pittsburgh | MEMS Dept.

1. PACKAGE PRELIMINARIES & BASIC INDEX / ITENSOR SETUP

Julia Command / Code Syntax	Description, Usage & Mathematical Concept

`using Pkg`
`Pkg.add("ITensors")`
`Pkg.add("ITensorMPS")`
	Installation: Standard Julia package manager commands to install core ITensors.jl and tensor network ITensorMPS.jl packages.

`using ITensors` `using ITensorMPS`	**Module Loading**: Load core tensor functionality (dense tensors, indices, contractions) and MPS/MPO algorithms (DMRG, TDVP).
`i = Index(d)` `i = Index(d, "tag1,tag2")`	**Create Index**: Defines an Index object of dimension *d* with optional tags for identification (e.g., i = Index(2, "Qubit,Site")).
`A = ITensor(array, i, j)` `A = ITensor(i, j, k)`	**Create ITensor**: Wrap a Julia array/matrix with named Index legs. Uninitialized call ITensor(i,j) creates a zero-filled tensor of size *dim(i) × dim(j)*.
`A = randomITensor(i, j, k)`	**Random ITensor**: Creates an ITensor populated with normally distributed random floating-point entries. Useful for testing & debugging.
`A[i => 1, j => 2] = 2.0` `val = A[i => 1, j => 2]`	**Element Access / Assignment**: Get or set individual tensor entries by specifying key-value index pairs idx => position.
`arr = Array(A, i, j, k)` `mat = Matrix(A, i, j)`	**Convert to Julia Array**: Converts an ITensor into a standard Julia dense Array or Matrix with explicit leg ordering specified by indices.

`inds(A)`
`commoninds(A, B)`
`commonind(A, B)`
	Index Inspection: inds(A) returns all indices of A; commoninds(A, B) returns vector of shared indices between A and B.

2. TENSOR CONTRACTIONS, ARITHMETIC & INDEX PRIMING

Julia Command / Code Syntax	Description, Usage & Mathematical Concept
`C = A * B` `C = contract(A, B)`	**Tensor Contraction**: Fundamental contraction operation. Automatically contracts (sums over) *all matching Index objects* shared between A and B. Unshared legs survive into C.
`C = 4.0 * A - B / 2.0` `D = A + 3.0im * B`	**Linear Combinations**: Scalar multiplication, addition, and subtraction of ITensors. Requires A and B to possess identical sets of indices.
`norm(A)`	**Tensor Norm**: Computes Frobenius norm $\sqrt{(\sum \lvert A_{i,j,\ldots} \rvert^2)}$ of the tensor entries.

`A_prime = prime(A, i)`
`A_prime = prime(A)`
`A_prime = A'`
	Prime Level Adjustment: Increases prime level of index i (or all indices if omitted). Used to distinguish input/output legs or prevent unwanted contractions.

`noprime(A)` `plev.(inds(A))`	**Reset / Check Primes**: noprime(A) resets prime levels to 0. plev.(inds(A)) queries prime levels of all tensor indices.
`δ = delta(i, j)` `δ = delta(i, j, k)`	**Kronecker Delta Tensor**: Diagonal identity tensor $\delta_{i,j}$. Contracting A * delta(i, j) replaces index *i* with index *j*.
`A_tr = A * delta(j, l)`	**Tracing Degrees of Freedom**: Contracting with a delta tensor across shared legs traces out degrees of freedom (useful for partial traces in quantum density matrices).

ITensors Contraction Paradigm

Key Concept in ITensors: Unlike standard matrix multiplication where contraction depends on memory ordering (rows/cols), ITensors contract strictly based on **Index identity** (matching Unique IDs and prime levels). Transposing or reordering indices in memory does NOT alter contraction results. Use prime() to keep legs distinct or delta() to substitute legs.

3. RESHAPING (COMBINERS), DAGGER & MATRIX DECOMPOSITIONS (QR/SVD)

Julia Command / Code Syntax	Description, Usage & Mathematical Concept
`C = combiner(i, j, k; tags="ijk")` `ci = combinedind(C)`	**Combiner Tensor**: Merges multiple tensor legs *i, j, k* into a single combined index ci of dimension $d_i \times d_j \times d_k$. Preserves total Hilbert space dimension.
`A_reshaped = C * A`	**Combine / Reshape Legs**: Contracting ITensor A with combiner C merges legs *i,j,k* into single leg ci (e.g., converting rank-4 tensor to matrix).
`A_orig = dag(C) * A_reshaped`	**Uncombine / Split Legs**: Contracting reshaped tensor with adjoint combiner dag(C) exactly restores original multi-leg structure without losing information.
`A_dag = dag(A)`	**Dagger / Complex Conjugate**: Returns complex conjugate of all tensor elements. Essential for bra vectors ⟨ψ|, adjoint operators $A^\dagger$, and leg uncombining.
`Q, R = qr(A, (i, j))`	**QR Decomposition**: Factorizes ITensor *A* into orthogonal isometric tensor *Q* (containing legs *i, j* and a new Link index) and upper-triangular tensor *R*.
`U, S, V = svd(A, (i, k);` `cutoff=1e-8, maxdim=10)`	**Singular Value Decomposition (SVD)**: Decomposes *A* into $U \cdot S \cdot V^\dagger$. Accepts truncation keywords: cutoff (truncation threshold) and maxdim (max bond dimension).
`err = (norm(U*S*V - A)` `/ norm(A))^2`	**SVD Truncation Error**: Truncation discards singular values below threshold. Relative sum of squared discarded singular values gives squared Frobenius error $\sum_{k=\chi+1} s_k^2 / \sum s_k^2$.

4. MATRIX PRODUCT STATES (MPS) CONSTRUCTION & SETUP

Julia Command / Code Syntax	Description, Usage & Mathematical Concept

`sites = siteinds("Qubit", N)`
`sites = siteinds("Electron", N)`
`sites = siteinds("S=1/2", N)`
	Physical Site Indices: Generates array of site indices with predefined physical Hilbert spaces (e.g., *Qubit* (d=2), *Electron* (d=4), *Spin-1/2* (d=2)).

`ψ = MPS(C, sites;` `cutoff=1e-8, maxdim=10)`	**MPS from Full Vector/Tensor**: Decomposes a full rank-N tensor *C* into an MPS chain of N 3-leg site tensors using sequential SVDs with specified truncation.
`ket = ["0", "1", "0", "1"]` `ψ = MPS(sites, ket)`	**MPS Product State**: Directly constructs an exact MPS product state |0101…⟩ from state strings. Bond dimensions are all $\chi = 1$.
`ψ = randomMPS(sites; linkdims=10)`	**Random MPS**: Creates a random MPS state with specified internal bond/link dimensions (default $\chi = 10$).
`linkdims(ψ)` `maxlinkdim(ψ)`	**Query Bond Dimensions**: linkdims(ψ) returns vector of link/bond dimensions χ_l across all bonds. maxlinkdim(ψ) returns $\max(\chi_l)$.

`ψ_sum = ψ1 + ψ2`
`ψ_sum = +(ψ1, ψ2;`
`cutoff=1e-8, maxdim=10)`
	MPS Addition / Superposition: Adds two MPS states $\|\psi_1\rangle + \|\psi_2\rangle$. Bond dimensions add up to $\chi_1 + \chi_2$; use cutoff or maxdim to truncate sum.

`norm(ψ)` `normalize!(ψ)`	**MPS Norm & Normalization**: norm(ψ) computes $\sqrt{\langle\psi\|\psi\rangle}$ via site-by-site contraction. normalize!(ψ) normalizes state in-place to $\|\psi\| = 1$.
`ψ_tr = truncate(ψ;` `cutoff=1e-2, maxdim=10)`	**MPS Truncation**: Compress bond dimensions of an existing MPS by sweeping through and truncating singular values below cutoff or capped at maxdim.

5. MPS OPERATIONS, OPERATORS & EXPECTATION VALUES

Julia Command / Code Syntax	Description, Usage & Mathematical Concept
`val = inner(φ, ψ)` `val = dot(φ, ψ)`	**MPS Overlap / Inner Product**: Efficiently computes $\langle\phi\|\psi\rangle$ by contracting conjugate MPS ϕ with ψ site-by-site in $O(N\, d\, \chi^3)$ time without forming full state vector.

`Op = op("Z", sites[n])`
`Op = op("X", sites[n])`
`Op = op("N", sites[n])`
	Construct Local Operator: Creates a 2-leg local operator ITensor (e.g., Pauli Z, X, Y, creation/annihilation $c^\dagger$, c, number operator N) acting on site index sites[n].

`ψ_after = apply(Op, ψ;` `cutoff=1e-8, maxdim=10)`	**Apply Operator to MPS**: Applies local single-site or two-site gate/operator *Op* to MPS state ψ, automatically compressing resulting MPS to specified bond dimension.
`evs = expect(ψ, "Z")` `evs = expect(ψ, "X")`	**Expectation Values**: Computes single-site expectation values $\langle\psi\|O_i\|\psi\rangle$ for all sites i = 1…N. Returns a Vector of length N.
`C = contract(ψ)` `mat = Array(C * comb, c_ind)`	**Contract MPS to Full Vector**: Contracts all MPS site tensors into a single full state tensor *C*. Use a combiner(sites) to convert back to standard Julia state vector.

6. SUMMARY OF KEY WORKFLOW PATTERNS

Core Execution Recipe

Standard Tensor Network Workflow Pattern:

1. **Define Hilbert Space:** sites = siteinds("Qubit", N)
2. **Initialize State:** ψ = MPS(sites, ["0", "0", ...]) or randomMPS(sites, linkdims=χ)
3. **Apply Gates / Evolution:** Z1 = op("Z", sites[1]); ψ = apply(Z1, ψ; cutoff=1e-8, maxdim=20)
4. **Compute Observables:** z_exp = expect(ψ, "Z") or overlap = inner(ψ1, ψ2)
5. **Control Bond Dimension:** Always pass cutoff (e.g., 1e-8) and maxdim (e.g., 100) to prevent exponential bond growth during MPS sums or gate applications.